

Backend — Faturamento, Estoque, Pacientes, WhatsApp e Pannel de Senhas

1. Faturamento (Guias Pendentes)

Módulo de controle de **guias pendentes de faturamento**, com banco de dados próprio, separado do banco SQL Server **SMART** da Píxeon.

Banco de dados: `guias.db` (SQLite)

- Arquivo local em `backend/guias.db`, acessado via `get_conn_guias()` (usa `sqlite3`, não `pyodbc`).
- Por quê separado: guias pendentes é um controle **interno do Dashboard** (lançado manualmente pela equipe), não existe estrutura equivalente no Smart/Píxeon — não faz sentido gravar isso no banco de produção da Píxeon.
- Tabela `guias_pendentes`, criada em `inicializar_db_guias()`:
- `id` (PK autoincrement), `data`, `paciente`, `os_serie`, `os_num`, `tipo_exame`, `valor`, `setor`, `convenio`
- `status` — `CHECK` restrito a `'Pendente'`, `'Entregue'`, `'Cancelada'` (constante `STATUS_GUIAS_VALIDOS` no Python espelha essa lista)
- `data_entrega`, `data_faturamento`, `observacao`, `criado_em`, `criado_por`, `atualizado_em`, `atualizado_por`
- Índices em `status` e em `(os_serie, os_num)`

Endpoints

- `GET /api/faturamento/buscar-os` — busca uma OS já lançada na produção real do Smart (tabelas `osm + smm + smk + pac + cnv`) pelo número (aceita `"serie-numero"` ou só o número), para autopreencher paciente/valor/setor/convênio ao lançar uma guia nova. Calcula o valor líquido por item: `SMM_VLR - SMM_VLR_DESCONTO - SMM_VLR_COPARTIC + SMM_AJUSTE_VLR`, somando os itens da OS. Também retorna os itens individuais, para permitir escolher só os serviços ainda pendentes quando a OS tem vários itens (alguns já faturados). Mapeia setor `"PSI"` → `"RCN"` manualmente.
- `GET /api/faturamento/guias` — lista guias no SQLite, com filtros opcionais `status`, `setor`, `q` (busca em paciente/tipo_exame), `ano + mes` (usando `strftime('%Y-%m', data)`).
- `GET /api/faturamento/resumo` — conta e soma `valor` por `status`, mais `pendentes_30dias` (guias com status `Pendente` há mais de 30 dias, calculado via `julianday('now','localtime') - julianday(data) > 30`).
- `GET /api/faturamento/dashboard` — dados agregados para gráficos: pendências dos últimos 12 meses por status (reestruturado em uma linha por mês, valor por status como colunas — formato esperado por gráfico de barras empilhadas do Recharts), top 10 por convênio e por setor (só guias `Pendente`).
- `POST /api/faturamento/guias` — cria uma guia nova (status inicial sempre `'Pendente'`), via modelo Pydantic `GuiaCreate`.
- `PUT /api/faturamento/guias/{guia_id}` — atualização parcial (só os campos enviados, via `model_dump(exclude_unset=True)`). Se o status for alterado para `Entregue` e não houver `data_entrega` definida (nem enviada nem já existente), preenche automaticamente com a data de hoje. Valida que o novo status esteja em `STATUS_GUIAS_VALIDOS`.
- `DELETE /api/faturamento/guias/{guia_id}` — remove a guia pelo id.

2. Estoque

Todos os endpoints usam o banco SQL Server `SMART` (tabelas `MAT` =materiais, `MMA` =movimentações, `LOT` =lotes, `GMM` =grupos de material).

- `GET /api/estoque/sintetico` — saldo por grupo de material (`GMM`), no estilo do relatório PDF Sintético da Píxeon: saldo do mês anterior (calculado somando/subtraindo movimentações anteriores ao período — `mma_e`), entradas, saídas e saldo atual (`MAT_QT_EST_ATUAL * MAT_VLR_PM`) por grupo, com totais gerais.
- `GET /api/estoque/analitico` — variante mais detalhada (não lida em profundidade nesta documentação).
- `GET /api/estoque/resumo` — KPIs gerais: total de itens (só com movimentação recente, para não distorcer com itens antigos/parados), itens com estoque, itens zerados, itens abaixo do ponto de ressuprimento, valor total em estoque (com quebra por curva ABC), movimentações do período (entradas/saídas em quantidade e valor) e contagem de lotes vencendo em 30/60/90 dias ou já vencidos.
- `GET /api/estoque/posicao` — posição atual item a item, com `status_estoque` calculado por CASE: `ZERADO` (qtd=0) → `CRITICO` (abaixo do ponto de ressuprimento) → `ATENCAO` (abaixo do ponto de segurança) → `EXCESSO` (acima do estoque máximo) → `NORMAL` . Também calcula `cobertura_dias = qtd_atual / consumo_medio` . Filtros: `curva` (A/B/C), `busca` (por descrição).
- `GET /api/estoque/giro` — **Giro de estoque** = saídas no período / estoque atual. **Cobertura em dias** = `estoque_atual * dias_do_periodo / saidas_do_periodo` . Junta `MAT` com movimentações agregadas do período via subquery.
- `GET /api/estoque/lotes-vencimento` — lotes com saldo > 0 vencendo nos próximos N dias (parâmetro `dias` , padrão 90) ou já vencidos, com `valor_em_risco = saldo * preco_medio` e status (`VENCIDO` / `CRITICO` ≤30d/ `ATENCAO` ≤60d/ `OK`).
- `GET /api/estoque/movimentacoes` — lista bruta de movimentações (até 200), filtrável por tipo (E=entrada, S=sáida).
- `GET /api/estoque/curva-abc` — distribuição de itens/valor por curva ABC, com percentual de valor de cada curva sobre o total.
- `GET /api/estoque/mov-por-dia` — série diária de valor de entradas/saídas, para gráfico de linha/barra.
- `GET /api/estoque/por-setor` , `GET /api/estoque/por-grupo` — variantes de agregação (não detalhadas aqui).

Todas as consultas filtram `MAT_DEL_LOGICA <> 'S'` (exclui materiais logicamente excluídos) e `MMA_IND_CANCELADA <> 'S'` (exclui movimentações canceladas).

3. Pacientes (módulo "Pacientes")

Backend dividido entre rotas `/api/pacientes/*` (métricas gerais) e `/api/pacientesdb/*` (análises geográficas/temporais mais elaboradas), todas contra o Smart.

- `GET /api/pacientes/resumo` — pacientes distintos atendidos (via `osm.osm_pac` distinto no período), novos cadastros (`pac.pac_dreg` — Data do Registro, campo correto conforme dicionário de dados da Píxeon, não confundir com outros campos de data), total da base (excluindo óbitos via `pac_dt_obito`), pacientes de retorno (mais de 1 OS no período) e total de faltas acumuladas (`pac.pac_falta`).
- `GET /api/pacientes/novos-por-semana` — novos cadastros por semana do ano (`DATEPART(week, pac_dreg)`).
- `GET /api/pacientes/faixa-etaria` — distribuição por faixa etária (0-17, 18-29, 30-44, 45-59, 60+), calculada com `DATEDIFF(year, pac_nasc, GETDATE())` .
- `GET /api/pacientes/por-sexo` — distribuição M/F/Não informado.
- `GET /api/pacientes/por-convenio` — pacientes atendidos por convênio (só convênios com `cnv_stat='A'`).
- `GET /api/pacientes/aniversariantes` — lista de aniversariantes do mês (padrão: mês atual), com telefone/celular/indicador de WhatsApp e data do último atendimento — usado para campanhas de contato.
- `GET /api/pacientes/top-atendimentos` , `GET /api/pacientes/servicos-por-sexo` , `GET /api/pacientes/servicos-comparativo` — variantes analíticas adicionais (não detalhadas aqui).

`/api/pacientesdb/*`

- `GET /api/pacientesdb/por-bairro` — agrupa pacientes por **bairro real**, não pelo campo de endereço bruto (`PAC_END` , que é texto livre digitado pela recepção). Usa uma função auxiliar `_rua_para_bairro()` (mapeamento baseado nos Correios) para normalizar rua → bairro, com fallback para o logradouro normalizado (`_normalizar_rua()`) quando não há mapeamento. Calcula também `pct_total` (percentual do total de pacientes do período) e agrupa `novos` / `retorno` por bairro. Filtra bairros irrelevantes (menos de 5 pacientes E sem nome de bairro identificado).
- `GET /api/pacientesdb/crescimento-base` — novos cadastros mês a mês no período (usa `pac_dreg`).
- `GET /api/pacientesdb/retorno-vs-novos` — comparativo mensal entre pacientes novos e pacientes de retorno.

4. Integração WhatsApp

Configuração (`main.py`)

- Config persistida em `whatsapp_config.json` (arquivo local, fora do banco Smart — mesmo padrão de `metas_config.json`), lida/escrita via `_load_wpp_config()` / `_save_wpp_config()` .
- Suporta 3 provedores, selecionáveis via campo `provider` : **wppconnect** (self-hosted, sem custo por mensagem — o usado em produção), **Z-API** e **Evolution API** (alternativas SaaS, mantidas no código mas não confirmadas como ativas).
- `GET /api/whatsapp/config` — retorna a config atual, mascarando tokens sensíveis como `****` .
- `POST /api/whatsapp/config` — salva nova config (campos em query string/form, não JSON body) e já atualiza as variáveis de ambiente do processo em memória (`os.environ[...]`), para valer imediatamente sem reiniciar o backend.
- `GET /api/whatsapp/grupos` — lista os grupos de WhatsApp que a sessão WPPConnect conectada já participa, via API `all-groups` do WPPConnect — só funciona com `provider=wppconnect` e sessão autenticada (QR code já lido).
- `POST /api/whatsapp/send-test` — dispara manualmente o envio do resumo (turno `manha` ou `fechamento`), útil para testar sem esperar o horário agendado.
- `GET /api/whatsapp/preview` — monta a mensagem (sem enviar) para conferência visual antes do envio real.

`whatsapp_sender.py` — geração das mensagens

Funções de busca de dados (todas recebem `query_func` , a função `query()` do `main.py` , injetada por parâmetro):

- `buscar_dados_manha()` — roda por volta das 07:00 (horário configurável). Busca: marcações do dia (excluindo canceladas 'C' e bloqueadas 'B' via `agm.agm_stat NOT IN ('C','B')`), vagas disponíveis (`EX_HORARIOS`), médicos por turno (manhã = antes das 12h, tarde = 12h em diante — um médico que atende nos dois turnos aparece nas duas listas, cada uma com sua própria contagem/horário), e o ticket médio dos últimos 30 dias — **exceto aos sábados**, quando usa só a média dos últimos 8 sábados anteriores (perfil de produção de sábado é bem diferente de dia de semana).
- `buscar_dados_fechamento()` — roda às 17:00 (ou 11:30 aos sábados, ver `scheduler.py`). Resultado final do dia.
- `buscar_dados_amanha()` — prévia da agenda do dia seguinte, enviada junto com o fechamento.
- `buscar_producao_hoje()` — produção acumulada de hoje vs. meta do dia, usado só para o aviso de "meta batida" (ver abaixo).
- `_calcular_metas()` — calcula a meta mensal, o percentual do dia (`meta_dia_pct` = média diária real / meta diária de dia de semana) e do mês (`meta_mes_pct` = produção acumulada / meta acumulada até hoje), e quanto falta para bater a meta mensal. Lê a configuração de `metas_config.json` (mesma usada pelo módulo Produção e Painel TV). **Considera feriados de Parauapebas-PA** (lista fixa: 1/1, 21/4, 1/5, 7/9, 12/10, 2/11, 15/11, 20/11, 25/12, mais Sexta-feira Santa, Corpus Christi — calculados via algoritmo de Gauss para a Páscoa —, 15/8 Adesão do Pará e 27/5 Aniversário de Parauapebas) e domingos como dias sem meta (meta=0); sábados usam `meta_sabado` (normalmente menor que a meta de dia de semana).

Funções de montagem de texto (`montar_manha` , `montar_fechamento` , `montar_previa_amanha` , `montar_meta_atingida`) transformam os dicionários de dados em texto formatado para WhatsApp (negrito com `*asteriscos*` , emojis, sem markdown real).

Envio e multi-provider

- `enviar_whatsapp_numero()` — despacha para `enviar_wppconnect`, `enviar_zapi` ou `enviar_evolution` conforme `WPP_PROVIDER`.
- `enviar_wppconnect()` — chama `POST {WPPCONNECT_URL}/api/{session}/send-message` com Bearer token. Distingue grupo (`@g.us`) de contato individual (`@c.us`) via `_normalizar_destino()` / `_eh_grupo()` (heurística: sufixo já presente no destino, ou 15+ dígitos após limpar caracteres não-numéricos = grupo). **Se a resposta for 401** (token expirado), chama `_wpp_regenerar_token()` automaticamente e tenta reenviar uma vez antes de desistir.
- `_wpp_regenerar_token()` — gera um token novo via `POST {base}/api/{session}/THISISMYSECURETOKEN/generate-token` (secretKey padrão do WPPConnect-server, não é segredo específico deste cliente), atualiza a variável de ambiente em memória e também persiste chamando `POST /api/whatsapp/config` do próprio Dashboard (loopback HTTP), para o novo token sobreviver a um restart do processo.
- `enviar_whatsapp()` — envia para uma lista de números (padrão: `DEST_NUMBERS` do `.env`), com 1 segundo de espera entre cada envio.
- `enviar_resumo(turno)` — função principal. `turno="manha"` envia uma mensagem; `turno="fechamento"` envia duas (fechamento + prévia de amanhã, com 3s de intervalo entre elas).
- `enviar_meta_atingida()` — verifica se a produção de hoje já bateu a meta do dia/sábado e, se sim, envia um aviso comemorativo (mensagem separada dos resumos de manhã/fechamento — mesmo evento que dispara o som/animação no Painel TV, mas por um caminho totalmente independente: aqui é polling do scheduler consultando o banco a cada 30s, lá é o frontend consultando o próprio endpoint de resumo).

`scheduler.py` — agendamento

- Roda em thread de background (`iniciar_scheduler_em_background()` , chamado pelo `main.py` no startup), com um `ThreadPoolExecutor` dedicado (`max_workers=2`) para isolar chamadas de rede do WPPConnect — existe um comentário no código relatando que uma chamada travada (`[Errno 22]` intermitente no Windows) já deixou a thread do scheduler travada por horas; agora cada chamada tem timeout de 60s via `_com_timeout()` , e se estourar, tenta de novo no próximo ciclo (a cada 30 segundos) em vez de travar o laço principal.
- **Horários configuráveis** (lidos de `whatsapp_config.json` a cada ciclo, sem precisar reiniciar): padrão 07:00 (manhã) e 17:00 (fechamento).
- **Regra especial de sábado:** turno de fechamento aos sábados sempre dispara às **11:30** (independente do horário configurado), refletindo o expediente mais curto.
- **Domingos:** nenhum envio (nem manhã, nem fechamento, nem meta).
- Cada envio só é marcado como concluído (`ultimo_envio[chave] = True`) se **todas** as mensagens daquele turno saírem com sucesso — uma falha transitória (ex: token expirado logo após um restart) não perde o dia, tenta de novo no próximo ciclo de 30s.
- Loop separado verifica a cada ciclo se a produção do dia já bateu a meta (`buscar_producao_hoje`) e dispara `enviar_meta_atingida()` uma única vez por dia, também só marcando como enviado após confirmação de sucesso.

5. Painel de Senhas (`/api/painel-fila/*`)

Backend do Painel TV de chamada de senhas — duas fontes de dados distintas, tratadas em endpoints separados:

- `GET/POST/DELETE /api/painel-fila/videos` — gerencia os vídeos institucionais exibidos no painel (upload/listagem/remoção de arquivos na pasta `_PAINEL_VIDEOS_DIR` , servida estaticamente em `/painel-tv/videos/`).
- `GET /api/painel-fila/prestadores` — lista as filas oficiais de senha cadastradas no Smart (`FLE_CFG_SENHA`), usada no seletor de configuração de qual fila aparece em qual TV. **Nota importante do código:** o campo `fle.FLE_STR_COD` (setor) não reflete de forma confiável a recepção física para esse tipo de fila (a maioria fica marcada como `'RPS'` independente do nome real da fila) — por isso a escolha de quais filas aparecem em cada TV é sempre manual, feita na tela de configuração do painel, não inferida automaticamente.
- `GET /api/painel-fila/senhas` — senhas chamadas pela recepção (guichês). O **guichê físico real** vem da tabela `MFL` (só gravada quando a config `FILA_CEGO=N` está ativa no `Smart.ini`), campo `MFL_LOC_ORIGEM_CHAMADA` , cruzado de volta para

fle por (MFL_FLE_DTHR_CHEG, MFL_FLE_STR_COD, MFL_FLE_PSV_COD) e depois para LOC (nome do guichê, ex: "Guichê 01"). Nem toda chamada tem esse registro (ex: chamadas feitas via totem) — nesse caso, cai no login de quem atendeu (FLE_USR_ATENDIMENTO) como alternativa. Uma mesma senha pode gerar mais de um registro em MFL (rechamada, reenvio para outro guichê) — a consulta usa ROW_NUMBER() particionado para pegar só a mensagem mais recente. Um comentário no código destaca que o filtro de data é feito por MFL_DTHR (início do índice clusterizado da tabela), não por MFL_FLE_DTHR_CHEG — filtrar pela coluna errada forçava varredura completa da tabela (~39 segundos de consulta).

- GET /api/painel-fila/status-senhas — contagem de "na fila"/"atendidos"/"preferenciais" e a próxima senha aguardando, por prestador — painel lateral.
- GET /api/painel-fila/pacientes — pacientes chamados diretamente pelo médico dentro do Smart (FLE_STATUS='X'), sem passar pela recepção/guichê — usa FLE_LOC_COD (que costuma ser nulo para esse tipo de chamada) com fallback para o nome do setor.
- GET /api/painel-fila/status-pacientes — equivalente de status por médico (contraparte de status-senhas , mas para o painel de pacientes).

6. Integração Clinia (agendamento via WhatsApp — externa)

Seção isolada, protegida por API key própria (header X-API-Key , não é o mesmo mecanismo de login de usuário do Dashboard):

- **Autenticação em duas camadas** (verificar_clinia_key): (1) o IP de origem da requisição precisa estar na lista CLINIA_ALLOWED_IPS (variável de ambiente) — **falha fechado**: sem essa lista configurada, ninguém passa mesmo com a key certa; (2) a API key enviada precisa bater exatamente (comparação hmac.compare_digest , resistente a timing attack) com CLINIA_API_KEY .
- GET /api/clinia/paciente/buscar — busca paciente por telefone, CPF ou nome (só leitura), para a Clinia confirmar identidade antes de oferecer agendamento.
- GET /api/clinia/agenda/disponibilidade — lista horários disponíveis para agendamento, mas **apenas vagas reabertas por cancelamento** (agm.agm_stat='C'). Limitação documentada no próprio código: a grade completa de horários por médico está codificada em bitmask na tabela AGD (campos AGD_MAT / AGD_VESP), que não é decodificada aqui — então esta consulta não enxerga a disponibilidade "em aberto" nunca ocupada, só o que já foi um agendamento e depois foi cancelado.
- **Fase 2 (criar agendamento pela Clinia) foi deliberadamente NÃO implementada** — o comentário no código explica que a tabela AGM tem 145 colunas e embute regras de negócio do aplicativo desktop da Pixeon (cálculo de valor por convênio, workflow de confirmação), e um INSERT direto ali tem risco real de duplicar horário ou gravar dado inconsistente sem mapear e validar isso primeiro em ambiente de homologação (smart_hml).